

BFS / DFS 뼈대 템플릿

공통 뼈대 비교

	DFS	BFS
자료구조	재귀 (Stack)	Queue
visited 체크 시점	방문할 때	큐에 넣을 때
활용	경로 탐색, 사이클	최단거리

인접 리스트 vs 인접 행렬

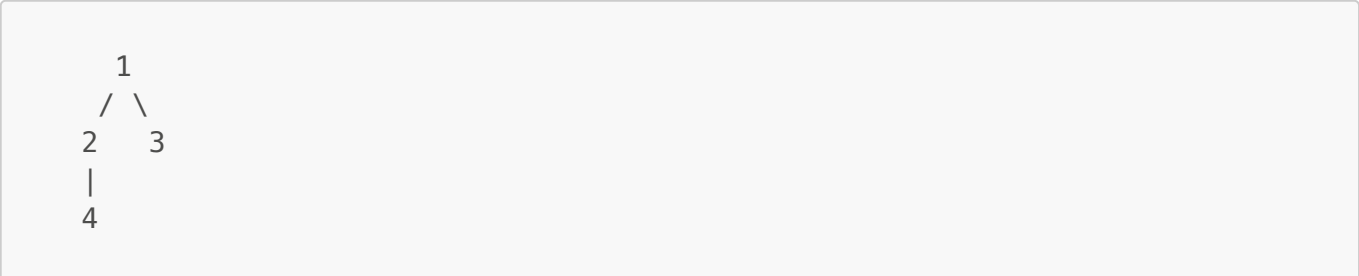
	인접 행렬	인접 리스트
공간	$O(N^2)$	$O(N+E)$
연결 확인	$O(1)$	$O(\text{degree})$
이웃 순회	$O(N)$	$O(\text{degree})$
코테 적합	노드 적을 때	대부분의 경우

N=1000이면 행렬은 100만 칸, 리스트는 실제 간선 수만큼만 → 리스트가 기본값

인접 리스트 자료구조 구조

```
List<List<Integer>> graph
  ↑ 외부 리스트      ↑ 내부 리스트
  인덱스 = 노드번호   값들 = 연결된 노드들
```

```
graph[0] → []      ← 0번 인덱스는 비워둠 (노드 1번부터 시작)
graph[1] → [2, 3]   ← 1번 노드는 2, 3과 연결
graph[2] → [1, 4]   ← 2번 노드는 1, 4와 연결
graph[3] → [1]       ← 3번 노드는 1과 연결
graph[4] → [2]       ← 4번 노드는 2와 연결
```



단방향 vs 양방향:

```
graph.get(a).add(b);           // a → b   (단방향)
graph.get(a).add(b);
graph.get(b).add(a);           // a ↔ b   (양방향)
```

이웃 꺼내는 방식 3가지 패턴

유형	이웃 꺼내는 방식	추가 조건
인접 리스트	<code>graph.get(node)</code>	없음
인접 행렬	<code>graph[node][next]</code>	<code>== 1</code> 체크
격자	<code>dx[], dy[]</code>	범위 체크

BFS/DFS 뼈대는 동일, "이웃을 어떻게 꺼내느냐"만 다르다

패턴 1 — 인접 리스트

```
static int n, m;
static List<List<Integer>> graph = new ArrayList<>();
static boolean[] visited;

// DFS
static void dfs(int node) {
    visited[node] = true;
    for (int next : graph.get(node)) {           // 이웃: 리스트에서 바로 꺼냄
        if (!visited[next]) dfs(next);
    }
}

// BFS
static void bfs(int start) {
    Queue<Integer> q = new LinkedList<>();
    visited[start] = true;
    q.offer(start);
    while (!q.isEmpty()) {
        int node = q.poll();
        for (int next : graph.get(node)) {       // 이웃: 리스트에서 바로 꺼냄
            if (!visited[next]) {
                visited[next] = true;
                q.offer(next);
            }
        }
    }
}
```

패턴 2 — 인접 행렬

```

static int n;
static int[][] graph; // graph[a][b] = 1이면 연결
static boolean[] visited;

// DFS
static void dfs(int node) {
    visited[node] = true;
    for (int next = 1; next <= n; next++) {
        if (graph[node][next] == 1 && !visited[next]) { // 이웃: 연결 여부 체크
            dfs(next);
        }
    }
}

// BFS
static void bfs(int start) {
    Queue<Integer> q = new LinkedList<>();
    visited[start] = true;
    q.offer(start);
    while (!q.isEmpty()) {
        int node = q.poll();
        for (int next = 1; next <= n; next++) {
            if (graph[node][next] == 1 && !visited[next]) { // 이웃: 연결 여부 체크
                visited[next] = true;
                q.offer(next);
            }
        }
    }
}

```

패턴 3 — 격자 (2차원 배열)

```

static int n, m;
static int[][] map;
static boolean[][] visited;
static int[] dx = {0, 0, 1, -1}; // 상하좌우 x이동
static int[] dy = {1, -1, 0, 0}; // 상하좌우 y이동

// DFS
static void dfs(int x, int y) {
    visited[x][y] = true;
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d];

```

```

        int ny = y + dy[d];
        if (nx >= 0 && nx < n && ny >= 0 && ny < m && !visited[nx][ny]) {
// 이웃: 범위 체크
            dfs(nx, ny);
        }
    }
}

// BFS
static void bfs(int sx, int sy) {
    Queue<int[]> q = new LinkedList<>();
    visited[sx][sy] = true;
    q.offer(new int[]{sx, sy});
    while (!q.isEmpty()) {
        int[] cur = q.poll();
        int x = cur[0], y = cur[1];
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d];
            int ny = y + dy[d];
            if (nx >= 0 && nx < n && ny >= 0 && ny < m && !visited[nx]
[nx][ny]) { // 이웃: 범위 체크
                visited[nx][ny] = true;
                q.offer(new int[]{nx, ny});
            }
        }
    }
}
}

```

풀 템플릿 (인접 리스트 기준)

```

import java.util.*;

public class Main {
    static int n, m;
    static List<List<Integer>> graph = new ArrayList<>();
    static boolean[] visited;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        n = sc.nextInt(); m = sc.nextInt();
        visited = new boolean[n + 1];

        for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());

        for (int i = 0; i < m; i++) {
            int a = sc.nextInt(), b = sc.nextInt();
            graph.get(a).add(b);
            graph.get(b).add(a); // 양방향
        }
    }
}

```

```
        dfs(1);
        Arrays.fill(visited, false);
        bfs(1);
    }

    static void dfs(int node) {
        visited[node] = true;
        System.out.print(node + " ");
        for (int next : graph.get(node)) {
            if (!visited[next]) dfs(next);
        }
    }

    static void bfs(int start) {
        Queue<Integer> q = new LinkedList<>();
        visited[start] = true;
        q.offer(start);
        while (!q.isEmpty()) {
            int node = q.poll();
            System.out.print(node + " ");
            for (int next : graph.get(node)) {
                if (!visited[next]) {
                    visited[next] = true;
                    q.offer(next);
                }
            }
        }
    }
}
```